CrashGuard: Simulation-Based Evaluation of an IoT Crash Detection and Emergency Alert System for Two-Wheeler Riders

Adheesh Garg[1], Ayan Gatani[1], Aastha Gupta[1],
Syamasudha Veeragandham[2,*]

[1,2]School of Computer Science and Engineering,
Vellore Institute of Technology, Vellore, Tamil Nadu, India – 632014

adheesh.garg2023@vitstudent.ac.in
ayan.gatani2023@vitstudent.ac.in
aastha.gupta2023@vitstudent.ac.in
[*]Corresponding author: syamasudha.v@vit.ac.in

Abstract

CrashGuard is a handlebar unit for motorcycles, made from an ESP32 board and an MPU-6050 six-axis sensor, that should notice a crash and raise an alert. It reads the sensor at 100 Hz. A finite-state machine declares a crash only after four things have happened in this order: the bike was being ridden; the acceleration spiked, or dropped sharply towards free fall; a strong rotation came within 750 ms; and the bike then lay tilted and still for 2 s. A light and a buzzer come on at that point. Unless the rider presses cancel within 15 s, one event goes to a cloud service. No prototype exists yet. Our evidence comes from generated motion data, from a public motorcycle dataset, and from running the unchanged firmware against simulated peripherals.

On 270 freshly generated events, CrashGuard rejected all 210 non-crash events and detected all 30 high-side crashes, but it missed 17 of the 30 low-side crashes, so its overall sensitivity is 71.7% (F1 = 0.835). This rule follows the staged structure of the motorcycle fall detector of Boubezoul et al. (2013). When run on the events this rule detected more crashes, 86.7% but this rule also raised 34 false alarms. McNemar's test shows that the two rules differ significantly with $\chi^2 = 13.40$ and $p = 0.00025$. Shortening the 2 s rest hold brings false alarms back (21 of them at 1.0 s). Lowering the rotation gate from 90 to 70°/s caught eight more low-side crashes in the development data with no false alarm, a result we have not yet checked on separate data. Of fourteen firmware tests three showed problems: the rest timer continues to count when sensor reads fail a failed cloud connection causes the main loop to stop for 5 seconds and an alert that happens while Wi-Fi is down gets lost forever. With a 120 mA load, at 5 V the battery model says the device will work for 12.64 hours. Code, data and scripts are released so that every experiment reported here can be repeated.

Keywords: IoT; ESP32; MPU6050; Motorcycles; Detection; Simulation; Alerts; Safety.

1 Introduction

A rider who falls from a motorcycle on an empty road may lie there for some time before anyone notices. A crash detector, on the bike could call for help faster. It's only worth installing if it stays quiet during normal riding. A device that beeps every time it hits a pothole will soon be turned off. The task is thus a detection problem with two kinds of error, and the two do not carry the same cost. A missed crash can leave an injured rider without help. A false alarm costs little each time, but frequent false alarms make the device useless. CrashGuard is our attempt at such a detector: an ESP32 and an MPU-6050 on the handlebar, running a small state machine. No board has been assembled yet. We test the design with generated motion data, with public motorcycle recordings, and by running the real firmware against simulated peripherals. Below we go through the earlier work paper by paper, since each one changed something in the design.

Boubezoul et al. [1] wrote one of the early fall detectors aimed specifically at powered two-wheelers (PTWs). Their main observation is that the lean of a motorcycle in ordinary riding, together with shocks from the road surface, makes a single threshold on acceleration unreliable. They detect a fall in stages: the start of the fall, the growth of the lean angle, and a period of rest after the fall. We kept the stages. Because CrashGuard has to run on an inexpensive handlebar unit, however, the stages are written as a small finite-state machine, with a ride-qualification step in front and a cancellation window behind. We compare our detector with a rule which follows the structure of their algorithm, and with two simpler rules, on exactly the same six-axis data.

A few years later the same group published recordings of real PTW falls and critical manoeuvres [2]. Data of this kind are rare, as crashing a motorcycle on purpose is expensive and dangerous. The falls are the positive examples, and the manoeuvres (hard braking, acceleration in a curve, a badly degraded track, and so on) are negative examples which test whether a detector can stay quiet. We take four falls and seven manoeuvres from this set and use them for a limited purpose: we check whether the signals cross the acceleration and rotation gates of CrashGuard, that is, we test the scale of our thresholds and not the full decision. All four falls cross both gates. The unit conversion, filtering and choice of time windows are published with the code.

Gelmini et al. [3] developed a crash detection algorithm for two-wheeled vehicles and tested it on experimental riding data. Their work matters to us chiefly as a lesson about coverage. Ordinary riding produces a wide variety of signals, and a detector which has been shown only a few convenient crash examples tells us little about its behaviour on the road. So our generator does not only make crashes. It also makes hard braking, sharp turns, potholes, speed breakers, a knock followed by recovery, and a parked bike that tips over; and it makes low-side and high-side crashes as two separate classes. Keeping them apart turned out to matter: in the fresh data every high-side crash is detected, and all 17 misses are low-side crashes.

In the literature on human falls, Sucerquia et al. [4] introduced SisFall, a dataset recorded with a device worn at the waist. Its signals come from the human body rather than a handlebar, so we do not use them. What we borrow is its design: it contains many everyday activities which are not falls, so a detector evaluated on it has to show that it stays quiet as well as that it fires. We follow the same principle. Seven of our nine scenario classes are negative, and the class counts are given with every result. On the fresh events CrashGuard rejects all 210 negatives, although this holds, of course, only within our signal model.

Casilari et al. [5] compared the public datasets for wearable fall detection and found large differences in sensor position, sampling rate, recorded activities and recording protocol. A sensitivity of, say, 95% measured on one dataset may say little about another. For a new study, the practical consequence is that the data must be described in enough detail for a reader to judge what the numbers mean. We have tried to meet that standard. The paper states the channel units and sampling rate, the random seeds, the noise and bias levels, the clipping limits, and the kinematic transformation inside the generator. No event is split between development, validation and test, and a second random stream checks whether the results repeat.

In a follow-up [6] Casilari et al. Trained convolutional networks, on public fall datasets and I see that Casilari et al. Noticed that scores moved a great deal depending on the repository. We read this as a warning about our own second dataset. A new seed does give new events, but the generator, the mounting position and the motorcycle model are the same as before. So we call it a fresh-seed evaluation and nothing more. The public PTW recordings answer another question (are our signals on the right physical scale?) and get a table of their own. Holding out whole vehicles or riding sessions would be a stronger test; we have not done it yet.

Musci et al. [7] ran recurrent networks (LSTMs) online on an annotated version of SisFall. An LSTM looks at a run of samples, not one sample at a time, and that fits our own view that the timing of events tells you most of what you need. Where we differ is in how the timing is written down. Instead of a learned model we use explicit timing rules: a correlation window between impact and rotation, a verification timeout, and a

hold time for the rest condition. One advantage of explicit rules is that the same C++ code runs in the offline replay and in the firmware, so the two cannot disagree. Another is that, when a fault is injected, the transition at which the behaviour went wrong can be read directly from the log.

Yu et al. [8] published KFall, a large dataset with temporal labels, intended for pre-impact fall detection. A pre-impact detector, for example one which triggers a wearable airbag, must act within a fraction of a second. Our device acts after the event and has a few seconds to spare. The two problems therefore call for different timing measures, and it matters where a delay measurement begins and ends. We measure the time from the first candidate sample to confirmation separately from the 15 s cancellation window which follows it. For the fresh events the median of the former is 2.33 s. The firmware tests check that the deadline at the end of the window is met to the millisecond.

Rodegast et al. [9] trained machine-learning models for motorcycle collision detection on simulated accidents and simulated normal riding. Their target is a passive safety system, which must fire quickly and must not fire by mistake. Our target is a warning which the rider can cancel, so a slower decision is acceptable, but nuisance alarms remain costly. Like them, we rely on simulation. We generated the fresh events only after freezing the rules, and every rule we compare is run on identical input. Whatever separates two rules in the confusion counts, missed crashes against false alarms, therefore comes from the rules themselves.

Leoni et al. [10] proposed an automatic eCall method for motorcycles based on vehicle dynamics. Their method handles a high-side crash, in which the vehicle rolls over fast, differently from a low-side crash, in which the motorcycle slides out more slowly. We ran into the same split. A gate set for a violent high-side roll can let a slow slide pass unnoticed, and that is exactly what happens to CrashGuard: it rejects every non-crash event, yet on the fresh set it catches only 43.3% of low-side crashes. This is the main weakness of the detector. We report it separately so that the overall accuracy of 93.7% is not read as uniform protection.

Tian et al. [11] used a multi-head attention transformer for fall detection with a sensor worn on the shin. They report a test sensitivity of 90.5% and an overall accuracy of 93.1%, and they discuss how class imbalance affects numbers of this kind. Their sensor sits on a human leg and ours on a handlebar, so the figures are not comparable, and we make no attempt to compare them. We cite the work as recent evidence of what learned sequence models can do. From it we also take the habit of reporting the class counts, F1 and sensitivity next to accuracy; with 210 negatives and only 60 positives, accuracy alone would flatter our detector.

Finally, Pradhan et al. [12] combined a dual-stream transformer with Kalman-filter fusion of inertial signals, and report an F1-score of 91.10% on SmartFallMM in a subject-wise evaluation. Two of its ideas seem worth adopting in later work, namely the fusion of complementary inertial signals and the holding out of whole groups (for us, vehicles or riding sessions) during evaluation. At present CrashGuard uses three hand-designed features and fixed rules. Its contribution lies elsewhere, in a light decision sequence whose every state can be observed and tested.

1.1 Research Gaps and Demonstrated Improvements

Identified Research Gaps

Taken together, the work above leaves three gaps that matter for a small device of the kind we build.

Gap 1: Thresholds on acceleration alone. A fixed threshold on the acceleration magnitude cannot both ignore hard road shocks (potholes, rumble strips, speed breakers) and detect slow, low-energy slides.

Gap 2: Accuracy says little about low-side crashes. When most test events are negative, a high overall accuracy can coexist with poor detection of one type of crash, typically the low-side slide in which the deceleration is gentle.

Gap 3: The alert chain is rarely tested. Most studies end with classifier metrics. I have seen that they do not look at how the device behaves when the sensor fails, when the rider cancels, when the cloud service does not respond or when the battery runs down.

Demonstrated Engineering Contributions

The staged decision is due to Boubezoul et al. [1]. What we add is the engineering needed to fit it into a small handlebar box, and a fairly thorough test of the result.

1. **Ride qualification.** A leaky bucket fills while the bike moves and drains at half speed when it stops. Impacts count only if the bucket has recently been full, so a parked bike that falls over, or a knock while standing, is not taken for a crash.
2. Local. Cancellation. LED. Buzzer first. Nothing goes out of the device until a set 15 seconds cancel window is over.
3. Comparison on the data. CrashGuard, a rule based on the structure of Boubezoul et al. [1]. Two other simpler rules all work on the same events and paired McNemar tests compare them.
4. **Threshold sweeps.** The tilt, rotation and rest-hold thresholds are changed one at a time across 29 settings to find out which of them affects the missed low-side crashes and which affects the alarms.
5. **Firmware fault tests.** Fourteen situations run the firmware against simulated peripherals. Three of them found bugs: one in the rest timer one in a blocking network call and a missing retry, after a network outage.

1.2 Research Objectives

1. To write and test a 100 Hz six-axis state machine for the handlebar that flags crash-like events, warns the rider on the spot and accepts a cancel press.
2. To compare CrashGuard with a rule following Boubezoul et al. [1] and with two simpler rules on the primary and the fresh-seed event sets, reporting confusion counts, sensitivity, specificity, F1, delay and paired McNemar tests.
3. To learn from sweeps of the rest-tilt, rotation and rest-hold thresholds which one drives low-side recall and which one drives false alarms.
4. To run the firmware with controlled virtual minors under the sensor, calibration, timing and communication faults, and to record the projected and the actual behaviour.
5. To publish the data, plots, circuit and architecture diagrams and power model, together with a plain account of the limitations.

2 Related Work

This section regroups the same sources by topic and states what each one does; the introduction has already explained how each affects our design.

Motorcycle fall and crash detection. Boubezoul et al. [1] gave a simple fall detection algorithm for PTWs, in which a fall is recognised in stages from the motion of the vehicle. The same group later released recordings of PTW falls and critical manoeuvres [2], which we use for an external check of signal scale. Gelmini et al. [3] studied crash detection for two-wheeled vehicles on experimental riding data and stressed how varied ordinary riding is. Rodegast et al. [9] trained classifiers on simulated motorcycle collisions for fast activation of passive safety systems. Leoni et al. [10] gave a dynamics-based eCall method, in which fast high-side motion and slower side slips are treated differently.

Human fall detection and its datasets. SisFall [4] contains waist-worn recordings of falls and of activities of daily living. Casilari et al. [5] compared such public datasets and showed how much they differ in sensor position, sampling and protocol, and in a later study [6] they showed that the results of convolutional networks depend on the dataset used. KFall [8] added synchronised temporal labels and benchmark algorithms for pre-impact detection.

Learned sequence models. Musci et al. [7] applied LSTM networks to online fall detection on an annotated version of SisFall. Tian et al. [11] trained a multi-head attention transformer on a shin-worn device, with class balancing and focal loss, and reached 90.5% test sensitivity and 93.1% accuracy. Pradhan et al. [12] fused inertial streams with a Kalman filter ahead of a dual-stream transformer; on SmartFallMM, with 21-fold leave-one-subject-out evaluation, they report 91.10% F1. None of these studies puts the sensor where we do, labels events as we do, or tests on the same population. Their numbers are background here, not competitors.

The cloud interface. Incidents are reported through Blynk. Its documentation on device templates and event logging [13, 14] fixes the device identity, the event code and the firmware call. None of this is detection research. It does decide where the controller ends. What's more, the firmware tests watch that boundary in three cases, namely the event is sent, the event is skipped, the connection times out, on the whole. Indeed,as,well,thecloudbegins,though.

Table 1 lists each source with its sensing domain and the design or evaluation choice it led to. It is not a ranking, since the sources were never run on a common dataset.

Table 1: Prior work and where it enters CrashGuard. The public PTW recordings are our external check on signal scale. The human-fall studies shaped how we evaluate, but their sensors sit elsewhere on the body and their labels differ.

Source	**Sensing domain / method**	**Relevance to this work**
Boubezoul et al. (2013) [1]	Motorcycle fall algorithm	Source of the staged decision. Our rules expose the candidate, rotation and rest states, so that each can be replayed.
Boubezoul et al. (2019) [2]	Public PTW recordings	Four controlled falls and seven critical manoeuvres. We keep their labels and check the acceleration and rotation gates separately.
Gelmini et al. (2021) [3]	Motorcycle crash detection	Reason for covering many vehicle behaviours; braking, turning, road shocks and recovery are explicit negative classes in our data.
SisFall (2017) [4]	Waist inertial dataset	Shows the value of everyday non-fall examples. We carry over the principle, not the human-body labels.
Casilari et al. (2017) [5]	Dataset analysis	Reason for reporting sensor location, sampling rate, units, event composition and the split procedure.
Musci et al. (2018) [7]	Online LSTM detection	A learned temporal model; CrashGuard writes the same kind of timing as explicit correlation and rest rules.
Tian et al. (2026) [11]	Shin-mounted transformer	Recent attention-based work on human falls, sensitive to class balance. The mounting differs from our handlebar.
KFall (2021) [8]	Pre-impact timing dataset	Separates pre-impact timing from post-event detection. We report the confirmation delay apart from the cancellation window.
Casilari et al. (2020) [6]	CNNs on several datasets	Reason for keeping the fresh random stream apart from the public PTW data, which come from another acquisition domain.
Rodegast et al. (2024) [9]	Simulated collisions, learned models	Fast activation of protective systems; our application is a warning which can be cancelled.
Leoni et al. (2025) [10]	Dynamics-based motorcycle eCall	Separates fast high-side motion from slower side slips, which is why we report low-side misses on their own.
Pradhan et al. (2026) [12]	Dual-stream inertial fusion	Kalman fusion and subject-wise evaluation; suggests vehicle-wise or session-wise evaluation for us.
Blynk documentation [14]	Device event interface	Defines the event code and description. We instrument the submission and do not equate it with a notification received on a phone.

Summary of Gaps Addressed

- **Choice of threshold.** If you set the acceleration threshold then slow low-energy falls will slip under it. If you set the threshold low then every rough patch of road will trigger it.
- **Scope of the evaluation.** For a safety device accuracy is the beginning. What happens after a real crash depends on calibration. It also depends on whether the rider can cancel. It also depends on the alarm deadline. It also depends on how the network behaves.
- **Openness.** If the detector code, the event sets and the fault traces stay private nobody else can check the numbers. Nobody else can find the bugs.

Problem Statement

The input is a 100 Hz stream of three-axis acceleration and three-axis angular rate measured on a motorcycle handlebar. While the bike is being ridden, check if the stream shows an impact or unloading a rotation that happens right after then a tilted rest that lasts. In that exact order. If it does turn on the outputs. Wait 15 seconds for a cancel signal. After that send one IoT event. The evaluation must count alarms and missed

falls. It should also measure how long it takes to detect the event. The detection delay.. It should show what happens when there are faults. All of this should be tested in experiments that other people can repeat.

3 Materials and Methods

3.1 Experimental platform and materials

The parts list is short. Controller is an ESP32-WROOM-32 development board. Sensor is a GY-521 breakout that holds the MPU-6050. Warnings are a LED behind a 220 Ω resistor and a 5-V passive piezo that is switched by a transistor. Input is one active-low push-button. Power is a 2600 mAh 18650 cell, a TP4056/DW01A charge-and-protection module and an MT3608 boost converter that is set to 5 V. In firmware the sensor sits at I^2C address `0x68` on a 400 kHz bus with a 44-Hz low-pass filter, a 100-Hz output and full-scale ranges of ±16 g and ±2000 °/s [15]. None of this has been. Measured. The schematic, the interface calculations and the discharge curves are design and simulation results. Both the detector header and the firmware sketch used in the experiments are the project's real source files [16]. For the motion data a native C++ program replays the detector event, by event. For the firmware a separate harness compiles the sketch untouched and links it against test doubles for the Arduino core, the Wire library, Wi-Fi and a Blynk-compatible service. Time comes from a clock and signed register values travel through the raw I^2C read path. The harness logs pin states, tone frequency, serial output, event payloads and connection delays. Whenever we write "the cloud" below we mean this service. Table 2 gives the pin for each function. The harness watches these pins so the same table ties the circuit to the code.

Table 2: ESP32 pin allocation. SDA and SCL are the I^2C data and clock lines. ADC1 is the first ADC unit. All signals use the ground. The battery row shows the divider that gets the cell voltage, from the ADC reading.

Element	ESP32 interface	Configuration	Function
MPU-6050	GPIO21 / GPIO22	SDA / SCL	Acceleration and angular-rate acquisition.
LED	GPIO26	Active high	Visible crash warning.
Piezo driver	GPIO25	2400 Hz tone	Audible warning.
Cancel button	GPIO27	Internal pull-up	Cancel or acknowledge an alert.
Battery divider	GPIO34 / ADC1	100 kΩ / 100 kΩ	Cell voltage measurement.
Wi-Fi	Station interface	Phone hotspot	Blynk SSL event path.

3.2 System architecture

In Figure 1 the data moves from left to right: sensor, decision on the ESP32, local warning and finally the remote event. The battery chain and the rider's button are, on the row. They are part of the path but do not take part in the decision..

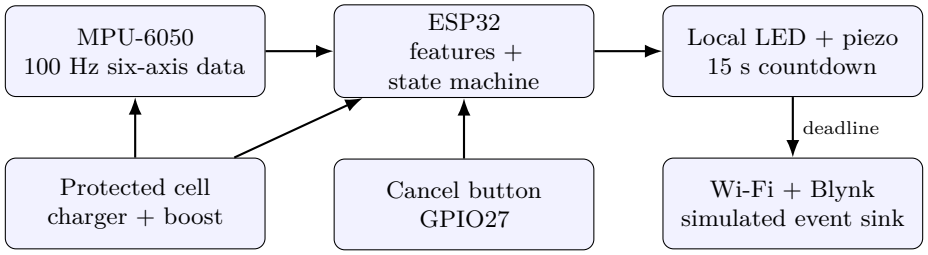


Figure 1: Architecture of CrashGuard: acquisition, local decision, cancellation by the rider, and the remote event. In the harness the event sink is replaced by an instrumented model of the service.

3.3 Circuit diagram and virtual-hardware image

Figure 2 gives the circuit at connection level: power conditioning, the sensor, the battery measurement and the warning outputs, with common nets marked by labels. The values of the components are the ones used in the DC interface calculation that follows and the diagram also shows how the transistor controls the piezo.

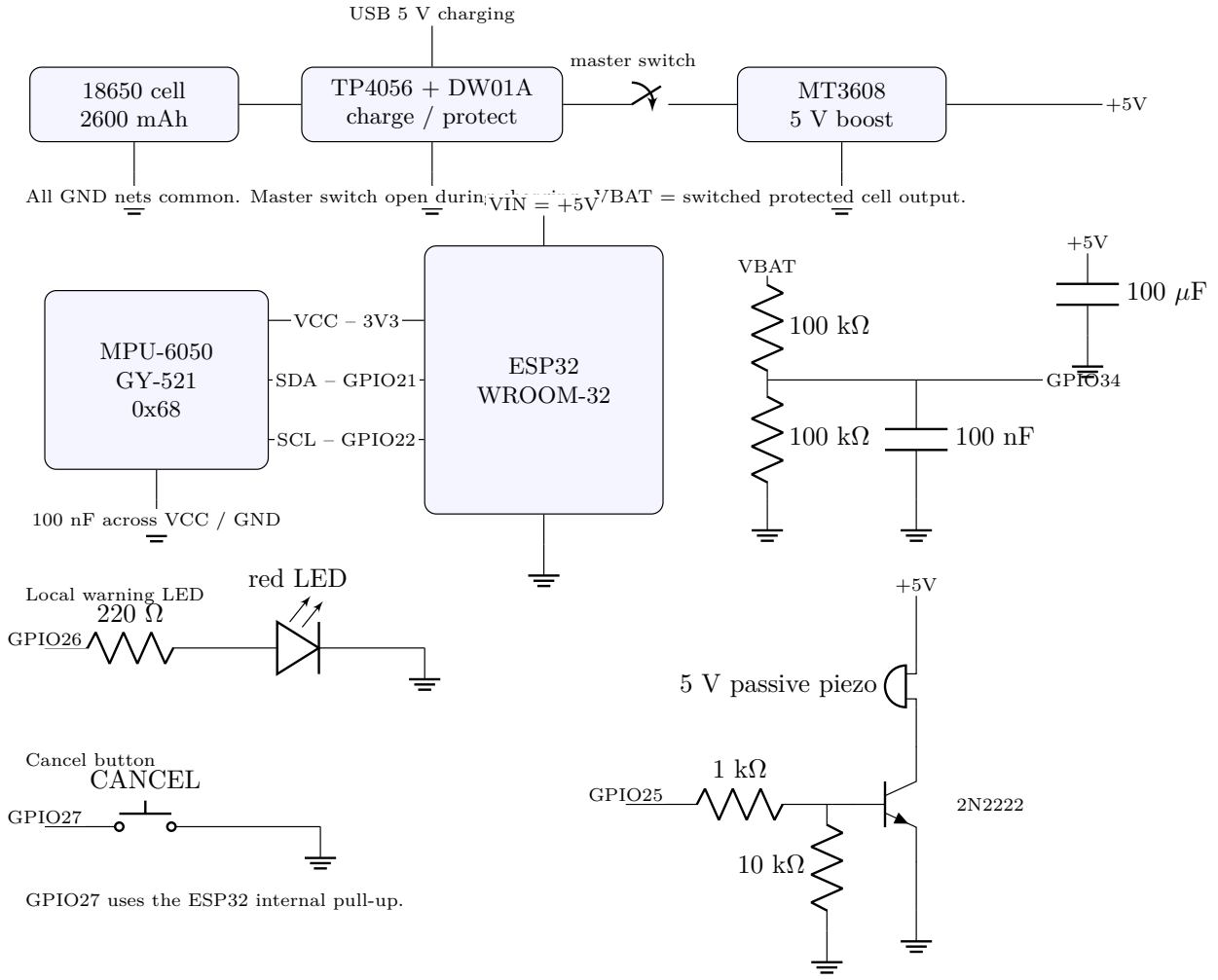


Figure 2: Connection-level circuit: power, sensing, battery divider, LED, transistor-driven piezo and cancel input. The master switch is open while charging because the charger module doesn't share the load.

The GY-521 module runs on 3.3 volts for both power and logic. GPIO25 controls the piezo through a 2N2222 transistor. There's a 1 kilohm resistor on the base and a 10 kilohm pull-down resistor. A 100 nanofarad capacitor connects from the middle of the voltage divider to ground. A 100 microfarad capacitor decouples the 5-volt rail. We did DC operating point calculations across 39 cases: thirteen different battery cell voltages from 3.0 to 4.2 volts each with an LED forward voltage of 1.8, 2.0 or 2.2 volts. In that grid the ADC pin sees a voltage between 1.50 and 2.10 volts. The voltage divider draws between 15 and 21 microamps. The LED carries between 5.0 and 6.82 milliamps. These are all values, not measured ones.. They have no connection to the battery experiment in Section 4.9.

Figure 3 is not a sketch of parts arranged for looks. It's a snapshot from one run captured exactly at the confirmation moment—3.610 seconds after the start. Every label, on the figure (LED level, tone frequency, countdown converted battery voltage) shows the value the harness recorded at that instant.

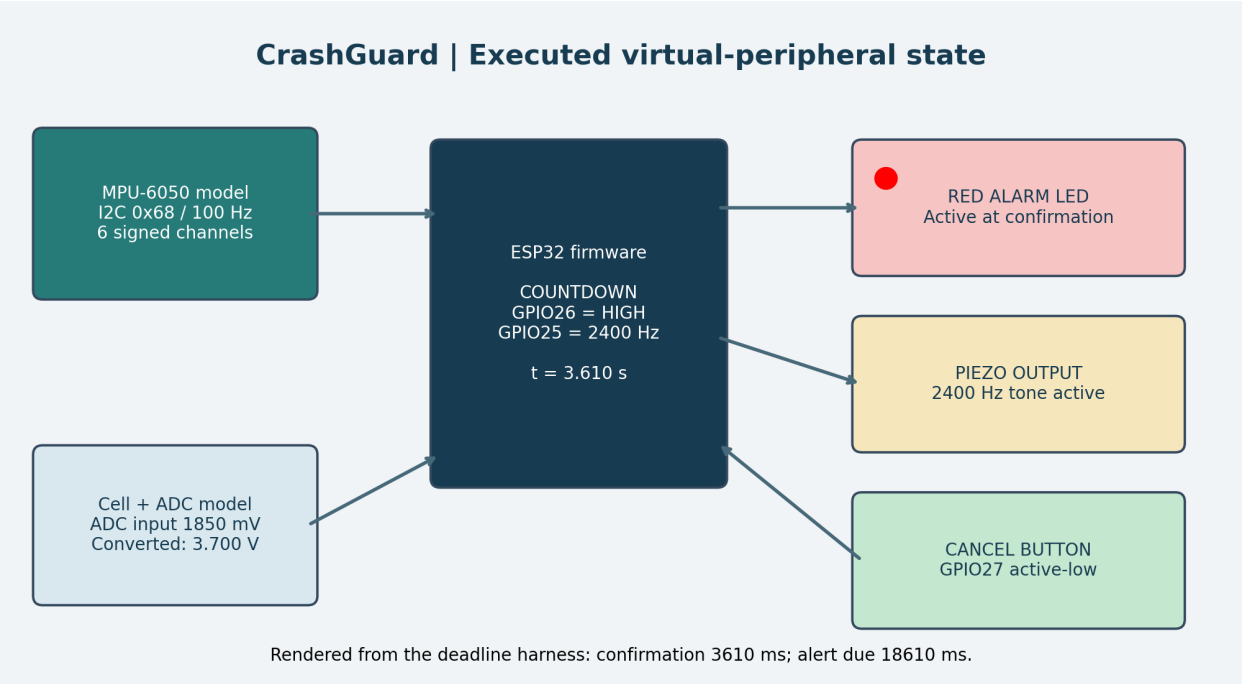


Figure 3: Harness state at t = 3.610 s, the moment of confirmation: LED on, piezo at 2400 Hz countdown started.

3.4 Dataset generation and preprocessing

The primary set holds 675 events, each lasting 10 seconds. At a sampling rate of 100 Hz this results in 675,000 six-axis samples. The data is spread across nine classes with 75 events, in each class. Normal riding, potholes, speed breakers, hard braking, sharp turning, a parked tip-over, and an impact followed by recovery are negative classes; low-side and high-side crashes are positive. This gives 150 positive and 525 negative events. The generator is fixed by the seed 20260828. A second seed, 20260911, gives a further 30 events per class, that is 270 fresh events and 270,000 samples.

Each event is built from roll, pitch and yaw profiles, the gravity vector in the body frame, pulses for the manoeuvre or impact, and noise, bias and clipping. Gravity is expressed in the body frame of the sensor as

$$\widehat{\mathbf{g}}_b = \begin{bmatrix} -\sin\theta \\ \sin\phi\cos\theta \\ \cos\phi\cos\theta \end{bmatrix} \in \mathbb{R}^{3\times 1}, \qquad \mathbf{g}_b = g_0\widehat{\mathbf{g}}_b, \qquad \|\widehat{\mathbf{g}}_b\|_2 = 1, \tag{1}$$

where ϕ and θ are the roll and pitch angles in radians, and the three entries are ordered along the x, y and z axes of the sensor (the subscript b stands for body frame). The hat marks a unit vector, so $\widehat{\mathbf{g}}_b$ is dimensionless and its entries can be read directly in units of g. Multiplying by standard gravity, $g_0 = 9.80665$ m/s^2, gives the physical vector $\mathbf{g}_b$ in m/s^2; g_0 is a constant and not a fitted parameter. For a level sensor, $\phi = \theta = 0$, the vector is $[0, 0, 1]^\mathsf{T}$. A positive roll gives a positive y component and a positive pitch a negative x component. Yaw does not appear, since rotation about the vertical does not change the direction of gravity.

Linear pulses representing the road surface, the rider's inputs and the impacts are then added to this gravity vector. To produce the gyroscope signals, the generator does not simply differentiate the Euler angles. It converts the angle rates $\dot{\boldsymbol{\Theta}} = [\dot\phi, \dot\theta, \dot\psi]^\mathsf{T}$ into body-frame angular velocities $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^\mathsf{T}$ with the standard Tait–Bryan Z-Y-X (yaw–pitch–roll) relation

$$\begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \mathbf{E}(\phi,\theta) \begin{bmatrix} \dot\phi \\ \dot\theta \\ \dot\psi \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\sin\theta \\ 0 & \cos\phi & \sin\phi\cos\theta \\ 0 & -\sin\phi & \cos\phi\cos\theta \end{bmatrix} \begin{bmatrix} \dot\phi \\ \dot\theta \\ \dot\psi \end{bmatrix}. \tag{2}$$

Here ψ is the yaw angle and $\mathbf{E}(\phi, \theta)$ is the 3×3 matrix which depends only on roll and pitch. When the motorcycle rolls and yaws at the same time, as in a high-side crash, the matrix mixes the angle rates across the three gyroscope axes, which is what a real body-fixed gyroscope would see. At zero roll and pitch the matrix is the identity, and the body rates are simply the angle rates.

The time of the event is drawn uniformly from 3.8 to 5.2 s. The acceleration bias has standard deviation 0.018 g and the sample noise 0.022 g; for angular rate the corresponding values are 0.8 and 1.5 °/s. Samples are clipped to the full-scale ranges of the sensor. The final roll angle lies between 48 and 105° for low-side crashes and between 70 and 135° for high-side crashes. Note that the low-side range deliberately reaches below the 55° tilt gate of the detector, so some low-side crashes cannot be detected by design. Impact pulses are between 3.0 and 11.5 g for high-side crashes and between 0.8 and 5.5 g for low-side crashes, before vector addition and clipping.

For the external signal analysis we load seven critical manoeuvres and four falls from the published PTW supplement [2]. We keep the finite numeric rows, filter the six channels against aliasing and resample them from 1 kHz to 100 Hz (polyphase resampling with a Kaiser window), convert acceleration from $\mathrm{m/s^2}$ to g by dividing by 9.80665, and keep angular rate in degrees per second. For the falls we cut 3 s windows centred at 40.132, 34.288, 35.876 and 43.486 s. The manoeuvres stay whole and count as negatives. Three seconds is enough to see whether a gate opens, but too short for the 2 s rest hold after the event, so these windows tell us nothing about the full decision.

3.5 Data split and evaluation protocol

The split is based on the order events were generated within each scenario. Events 1 through 45 go to development 46 through 60 to validation and 61 through 75 to test. That makes 405 events for development 135 for validation and 135 for test. Which's 60%, 20% and 20% respectively. No event gets split across subsets. Every assignment is saved in `results/splits.csv`.

There's nothing to train or fit here. CrashGuard is a set of fixed rules. No weights, no learning. The split only cuts one replay into pieces after the fact. Its "test" part isn't held out like in machine learning. The fresh set is different. It has 270 events, all generated after we locked the detector for this study. We didn't change any thresholds after looking at them. These still come from the generator just with a new seed. The public PTW recordings are kept for gate analysis. The human-fall datasets are, for context.

3.6 Features, calibration, and decision equations

Calibration comes first. The firmware takes 200 samples while the bike is standing still. If the mean acceleration has a norm outside 0.85 to 1.15 g or if the gyroscope RMS is above 5 degrees per second the whole batch gets thrown out. If not the mean acceleration is saved as the reference gravity vector. Also the three gyroscope means are saved as bias offsets. Let $\mathbf{a} = [a_x, a_y, a_z]^\mathsf{T}$ be the acceleration in g. Let $\mathbf{r} = [r_x, r_y, r_z]^\mathsf{T}$ be the reference gravity vector, in g. Let $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^\mathsf{T}$ be the rate in degrees, per second.Three scalar features are built from these. The first one starts with the acceleration magnitude.

$$A = \|\mathbf{a}\|_2 = \sqrt{a_x^2 + a_y^2 + a_z^2}, \tag{3}$$

with a_x, a_y and a_z taken at the same instant and divided by g_0. So $A = 1$ is one g, and the thresholds 2.0 and 0.35 below read as $2g$ and $0.35g$. On the device the raw signed counts are divided by 2048 counts per g (the $\pm 16g$ scale); a fresh sample comes in every 10 ms.

For rotation we take the norm of the angular rate,

$$\Omega = \|\boldsymbol{\omega}\|_2 = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2}, \tag{4}$$

where ω_x, ω_y and ω_z are bias-corrected rates about the sensor axes in degrees per second. Raw counts are divided by 16.4 per °/s (the ±2000°/s scale), and then the calibration means are subtracted. Thanks to the norm, it does not matter about which axis the bike mostly turns, or in which direction. Two thresholds are applied to Ω later, 90°/s for the rotation gate and 30°/s for rest; neither is part of the formula.

Finally tilt, the tilt angle is the angle that shows how the current acceleration vector differs from the reference.

$$\Theta = \frac{180}{\pi} \arccos\left[\mathrm{clip}\left(\frac{\mathbf{a}^\mathsf{T}\mathbf{r}}{\|\mathbf{a}\|_2 \|\mathbf{r}\|_2}, -1, 1\right)\right], \tag{5}$$

Here $\mathbf{a}^\mathsf{T}\mathbf{r}$ equals $a_x r_x + a_y r_y + a_z r_z$. The variable $\mathbf{r}$ equals the mean of the 200 accepted calibration samples. The function $\mathrm{clip}(u, -1, 1)$ equals $\min(1, \max(-1, u))$. After dividing by the two norms the result equals a

cosine. Round-off errors can push the result beyond ± 1, where arccos is undefined. That is why the clip function is used. The factor 180 divided by π converts the result to degrees giving Θ in the range from 0, to 180.If the product of the norms is below 10^{-6} the implementation returns a tilt of zero. Since Θ is computed from the instantaneous acceleration, it measures the lean of the vehicle properly only when the motion is small, that is, during the rest after a fall, which is where we use it.

With these features, the decision proceeds as follows. Riding activity is present when $|A - 1| \geq 0.08$ or $\Omega \geq 4^\circ/\text{s}$. Evidence of activity accumulates up to 1000 ms and decays at half that rate when there is no activity, and while the evidence is full a 5000 ms deadline for active riding is refreshed. Once riding is qualified, a candidate starts when $A \geq 2.0$ or $A \leq 0.35$. The rotation gate then needs $\Omega \geq 90^\circ/\text{s}$ no more than 750 ms away from the candidate. After that the detector waits for the rest condition

$$R = (\Theta \geq 55^\circ) \wedge (0.70 \leq A \leq 1.30) \wedge (\Omega \leq 30^\circ/\text{s}), \tag{6}$$

with $\wedge$ the logical AND. In words: the bike leans hard, the accelerometer reads $1g$ give or take $0.30g$, and there is almost no rotation. All comparisons include equality. The predicate is checked sample by sample; how long it has to stay true is decided by the state machine, not by Equation (6).

Once R has stayed true for 2000 ms the crash is confirmed. The try is dropped. So, memory use and work per sample are both constant, if that hasn't happened 6000 ms after the candidate, at least in practice. Indeed, confirmation is switching on the alarm and opens the 15,000 ms cancel window.

3.7 Algorithm and flowchart

Table 3 walks through the firmware in execution order, from sensor start-up to the one event it may send, and marks where a sensor or network failure takes it off the normal path, on the whole.

Table 3: Algorithm 1: firmware procedure. A failed sensor read skips the detector update. So, sampling period 10 ms. Calibration is employing 200 samples, in real-world scenarios, and step 8 is recording only what the service returned, not whether anyone was notified, in the main.

Step	Operation
1	Configure the GPIO pins and the MPU-6050; stop safely if the sensor does not answer.
2	Collect 200 samples; if the device moved or the data are invalid, repeat until a valid reference is obtained.
3	Start Wi-Fi association in the background and set up the 10 ms sampling schedule.
4	Check for a press of the cancel button; cancel a running countdown, or acknowledge an alert and silence the outputs.
5	At each sampling deadline read the 14-byte sensor burst; on failure log a warning and skip the detector update.
6	Compute A, Ω and Θ; update the activity evidence and the candidate, rotation and rest states.
7	On confirmation set the LED high, start the 2400 Hz tone and record the start of the countdown.
8	Run one event operation and record whether it was turned in, disabled, skipped for lack of Wi-Fi, or skipped for lack of cloud, when 15 s have elapsed, at least empirically.
9	Keep the local warning on until the rider acknowledges it; continue sampling.

The flowchart in Figure 4 shows the same procedure as a sequence of decisions, with its two return paths: a rejected candidate returns to sampling, and a cancellation by the rider stops the warning before the event falls due.

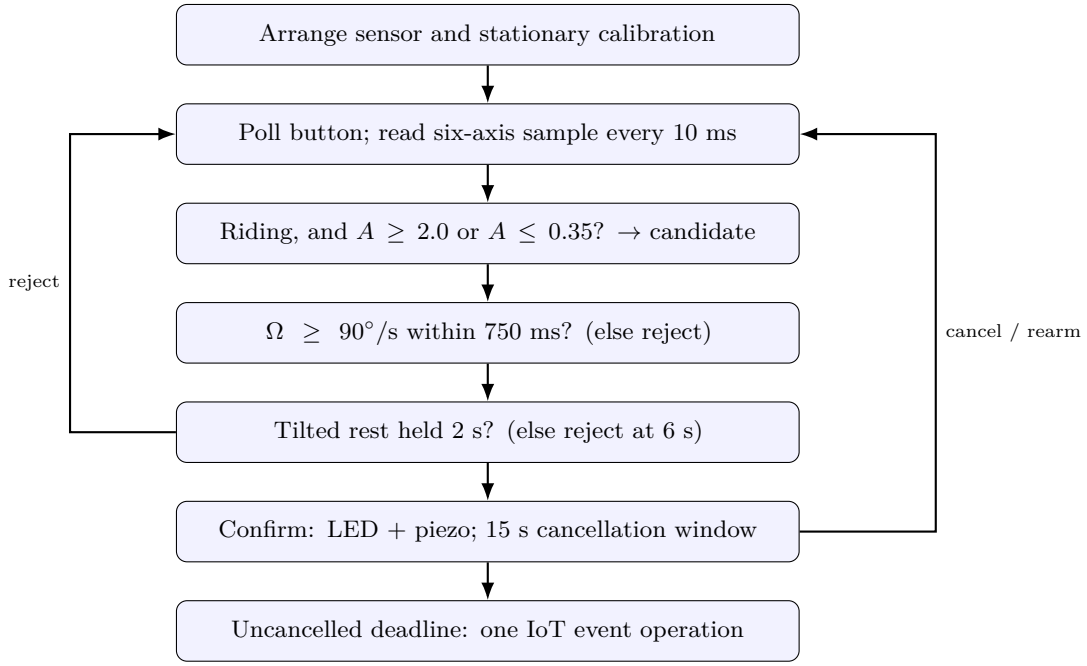


Figure 4: Flowchart of a successful detection, with the return paths for rejection and cancellation. Failed sensor reads also go back to sampling.

3.8 Baselines and metrics

Three other rules serve as baselines. The *acceleration-only* rule fires as soon as $A \geq 2.0$. The *strict ordered* rule needs $A \geq 4.0$, then $\Omega \geq 200°/s$ within 500 ms, and then a tilt of at least 60° with $0.70 \leq A \leq 1.30$ held for three seconds, all within eight seconds of the impact. The third rule follows the staged structure of the motorcycle fall detector of Boubezoul et al. [1], and we call it the Boubezoul rule. Stage one opens a fall when the roll rate $|\omega_x|$ reaches $60°/s$, or when $A \geq 2.5$ or $A \leq 0.40$. In stage two the lean must reach $\Theta \geq 55°$ within 2 s, otherwise the rule resets. In stage three the rest condition $\Theta \geq 55°$, $0.70 \leq A \leq 1.30$, $\Omega \leq 30°/s$ must hold continuously for 1.5 s, and the attempt is abandoned 6 s after the start. To be clear, these thresholds are our own choices, written to follow the structure of the published method; we do not claim that they are the exact values of the original paper, so this rule is an approximation of their detector and not a copy. Unlike CrashGuard, it has no ride qualification.

Let TP, TN, FP and FN be the numbers of true positives, true negatives, false positives and false negatives. All of them are counts of whole events, and in every experiment below the denominators of the metrics are positive. The first metric is accuracy,

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{7}$$

the fraction of all events which are classified correctly. The denominator is the size N of the evaluation set, $N = 675$ for the primary set and $N = 270$ for the fresh set. As there are many more negative events than positive ones, accuracy can be high even when many crashes are missed, which is why we never report it alone.

Sensitivity (also called recall, or the true-positive rate) is the fraction of crashes which are detected,

$$\text{Sensitivity} = \frac{TP}{TP + FN}. \tag{8}$$

The denominator is the number of labelled crashes, 150 in the primary set and 60 in the fresh set. This is the metric which shows missed crashes directly, and we also give it separately for low-side and high-side crashes. For example, the fresh CrashGuard result is $43/(43 + 17) = 0.717$, that is 71.7%.

Specificity is the fraction of non-crash events which are correctly rejected,

$$\text{Specificity} = \frac{TN}{TN + FP}, \tag{9}$$

with 525 negatives in the primary set and 210 in the fresh set. Specificity measures nuisance alarms, but only over the negatives we decided to generate; 100% means less if those negatives are few or all alike. Right so the gathered value is 1 for CrashGuard on the fresh set TN = 210 and FP = 0, in real-world scenarios. Two hundred and ten events cannot prove perfect rejection, which is why a Wilson interval is given below as well.

Precision asks how many of the alerts were real crashes:

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{10}$$

Its denominator is every event the rule called positive, whatever the truth, so precision rises and falls with how typical crashes are in the test set, in real-world scenarios. A rule that never fired would give 0/0. Our code returns zero in that case. Even as, it does not surface anywhere in this paper. For CrashGuard on the fresh set, precision is $43/(43 + 0) = 1$.

F1 puts precision and sensitivity together:

$$F_1 = \frac{2TP}{2TP + FP + FN}, \tag{11}$$

which is the harmonic mean of the two written in terms of counts; the subscript 1 means that both are weighted equally. F1 is penalised by both kinds of error, and we report it as a decimal between 0 and 1, whereas accuracy, sensitivity and specificity are given in percent. For the fresh CrashGuard counts, $F_1 = 86/103 = 0.835$.

The false-positive rate is

$$\text{FPR} = \frac{FP}{TN + FP}, \tag{12}$$

the fraction of non-crash events which raise an alarm, which is one minus the specificity. For the acceleration-only rule on the fresh set, $FP = 108$ and $TN = 102$, so FPR $= 108/210 = 51.4\%$; for CrashGuard the numerator is zero. Note that this is a rate per event. A rate of false alarms per hour of riding would need the riding time as its denominator, which our data do not provide.

The delay of a detection is the time of confirmation minus the time at which the successful candidate started, and it is computed only for detected crashes. The acceleration-only rule has, by definition, no confirmation hold and thus zero delay. For a proportion $\hat{p} = x/n$ with x successes out of n events, we give 95% Wilson intervals [17],

$$\begin{aligned} p_{\mathrm{L}} &= \frac{\hat{p} + z^2/(2n) - z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}, \\ p_{\mathrm{U}} &= \frac{\hat{p} + z^2/(2n) + z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}, \end{aligned} \tag{13}$$

where p_{L} and p_{U} are the lower and upper limits of the interval, and $z = 1.959964$ is the 97.5th percentile of the standard normal distribution. For sensitivity $x = TP$ and $n = TP + FN$, and for specificity $x = TN$ and $n = TN + FP$. The constants 2 and 4 come from the Wilson score construction. Unlike the simple normal interval, the Wilson interval stays informative when all observed events are successes, as happens for our specificity. The intervals describe uncertainty due to the finite number of events; they say nothing about whether the generator resembles real riding.

Finally, to test whether two rules differ on the same events, we use McNemar's test with Edwards' continuity correction [18],

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}, \tag{14}$$

where b is the number of events which CrashGuard classifies correctly and the other rule does not, and c is the number of events for which the opposite holds. Events on which the two rules agree do not enter the statistic. Under the null hypothesis that both rules have the same error rate, χ^2 follows a chi-squared distribution with one degree of freedom. When there are few discordant events ($b + c < 25$) we use instead the exact two-sided binomial p-value, $p = 2\sum_{k=0}^{\min(b,c)} \binom{b+c}{k} 0.5^{b+c}$.

4 Experimental Results and Discussion

4.1 Experimental conditions

The C++17 programs are compiled with optimisation and strict warnings. Python with NumPy, SciPy and Matplotlib generates the data, replays the detector, computes the metrics and draws the plots. Every event is usually performed by a freshly reset detector. In fact, inside the firmware harness are a virtual clock, a 14-byte register buffer, instrumented GPIO and tone outputs, and a local Blynk-compatible sink, all things considered. On request it hands the unchanged firmware a failed probe, a shortened read, a dropped link or a cloud timeout, and is needing no network credentials, at least in practice.

A rerun on the 675 primary events reproduced the earlier confusion counts exactly. In turn, adding the 270 fresh events brings the total to 945 events, or 945,000 samples. That aside these, we ran 14 firmware scenarios, 39 DC operating points and nine battery-discharge cases. These experiments test diverse layers of the framework, in many respects. As, well, we report their counts separately.

4.2 Original and fresh performance comparison

The four rules are being compared by Table 4 on the 675 primary events, so as well, its confusion counts are showing where each score comes from, in actual application. Far more non-crash events than the acceleration-only rule and the Boubezoul rule are being rejected by CrashGuard, on the whole. Till then, many more crashes are lost by the strict ordered rule, which also is rejecting every negative event.

Table 4: Four rules on the same 675 primary events (seed 20260828; 150 crashes, 525 non-crash), at least in practice. Counts are, more precisely, whole events. Acc. Sens. and Spec. All rows see same input, so any difference comes from the rule. in percent. F1 in [0, 1].

Detector	TP	TN	FP	FN	Acc. (%)	Sens. (%)	Spec. (%)	F1
Acceleration-only	134	236	289	16	54.8	89.3	45.0	0.468
Strict ordered	39	525	0	111	83.6	26.0	100.0	0.413
Boubezoul et al. (2013)	135	436	89	15	84.6	90.0	83.0	0.722
CrashGuard (Ours)	114	525	0	36	94.7	76.0	100.0	0.864

The comparison is being repeated by Table 5, with the rules unchanged, on the 270 fresh events, at least empirically. What we need to ask is, as it stands, whether the pattern seen on the primary set survives on a new random stream; no threshold was chosen by looking at these results.

Table 5: The unchanged rules on 270 fresh events (seed 20260911; 60 crashes and 210 non-crash events). Columns as in Table 4. This tests repeatability within the same generator. It isn't a validation on other vehicles or road sessions, in real-world scenarios.

Detector	TP	TN	FP	FN	Acc. (%)	Sens. (%)	Spec. (%)	F1
Acceleration-only	52	102	108	8	57.0	86.7	48.6	0.473
Strict ordered-threshold	12	210	0	48	82.2	20.0	100.0	0.333
Boubezoul et al. (2013)	52	176	34	8	84.4	86.7	83.8	0.712
CrashGuard (Ours)	43	210	0	17	93.7	71.7	100.0	0.835

On the primary set CrashGuard removes all 289 false alarms of the acceleration-only rule, and misses 20 more crashes than it. Against the Boubezoul rule, CrashGuard removes 89 false alarms (specificity 100.0% against 83.0%) and misses 21 more crashes (sensitivity 76.0% against 90.0%). The same pattern appears on the fresh set: CrashGuard has none of the 34 false alerts of the Boubezoul rule, which is raising specificity from 83.8% to 100.0% and precision from 84.4% to 93.7% (F1 0.835 against 0.712), on the whole, and yet, it misses 9 more low-side crashes (71.7% against 86.7%). Alright so neither rule wins on both counts. CrashGuard pays for its silence on non-crash events with missed slow crashes, and whether that is the right trade depends on how riders react to repeated false alarms, a question our data cannot answer.

McNemar's test validates that CrashGuard and the Boubezoul rule differently err: $\chi^2 = 40.81$ ($p = 1.68 \times 10^{-10}$) on the primary set and $\chi^2 = 13.40$ ($p = 0.00025$) on the fresh set, to a significant degree. The test should, however, be read with care. On the fresh set the discordant events are 34 non-crash events which only the Boubezoul rule gets wrong and 9 crashes which only CrashGuard misses, so the significance reflects mainly the false alarms of the Boubezoul rule; it is not evidence that CrashGuard is better at detecting crashes. Against the strict ordered rule, CrashGuard detects significantly more crashes without any extra false alarm ($\chi^2 = 73.01$, $p < 10^{-15}$ on the primary set; $\chi^2 = 29.03$, $p = 7.1 \times 10^{-8}$ on the fresh set). Against the acceleration-only rule the difference is larger still ($\chi^2 = 232.44$, $p < 10^{-15}$ on the primary set; $\chi^2 = 82.09$, $p < 10^{-15}$ on the fresh set).

On the fresh set the 95% Wilson interval for sensitivity is 59.2–81.5% and the observed specificity of 100.0% has the interval 98.2–100.0%. On the primary set the intervals are 68.6–82.1% for sensitivity and 99.3–100.0% for specificity. Observing no false alarms in a finite set of negatives does not imply that none would occur under every riding condition.

Table 6 gives the CrashGuard counts for each subset of the data. It keeps apart the retrospective development, validation and test breakdown of the primary set and the fresh-seed experiment, which play different roles.

Table 6: CrashGuard confusion counts and F1 for each whole-event subset. The primary simulation is split within each of the nine scenarios into 45 development, 15 validation and 15 test events (405/135/135 in total). The fresh-seed set has 30 further events per scenario. TP, TN, FP and FN add up to the number of events. No weights are fitted in any subset, since the rules are fixed.

Partition	Events	TP	TN	FP	FN	F1
Development	405	67	315	0	23	0.854
Validation	135	23	105	0	7	0.868
Retrospective test	135	24	105	0	6	0.889
Fresh-seed test	270	43	210	0	17	0.835

The F1 values of the three primary subsets lie between 0.854 and 0.889, and the fresh value, 0.835, is a little below them. Differences of this size are what one would expect from subsets with 30 positives or fewer.

4.3 Scenario analysis and graphs

Broken down by class, the results are very uneven. Every negative class is rejected without exception in both sets, and every high-side crash is caught (75/75 primary, 30/30 fresh). Low-side crashes do badly: 39/75 and 13/30. All 36 primary misses and all 17 fresh misses are, in effect, therefore low-side crashes. Afterwards on to the high-side profiles, the low-side ones hit softer, move more slowly and end with less lean, and fixed gates only catch the upper part of that range, in the main.

Figure 5 plots the per-class share of correct decisions for both sets. In turn, plus, a person accuracy figure averages that away, at least empirically. The low-side bar stops at 52% (primary) and 43% (fresh). 100% is being reached by eight bars.

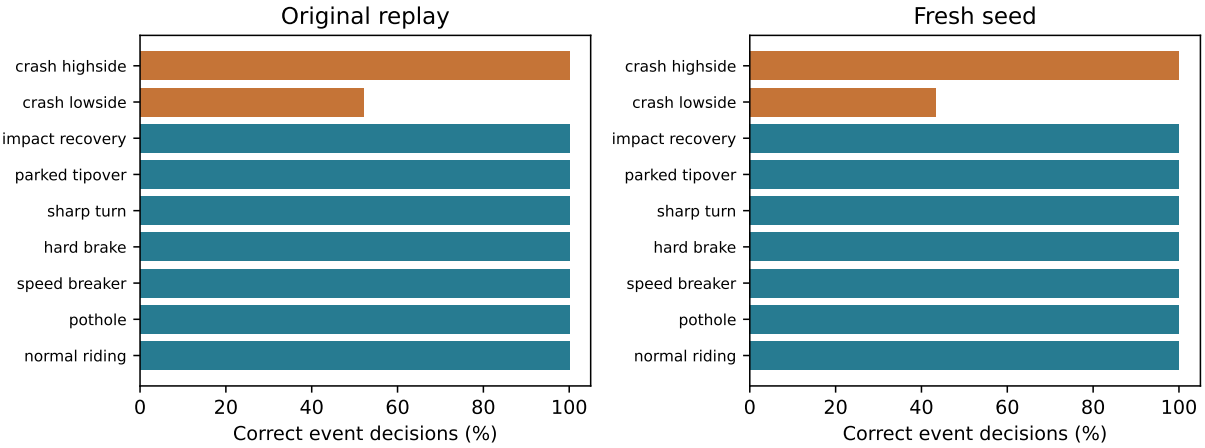


Figure 5: Correct event decisions per scenario: primary replay (left, 75 events per class) and fresh seed (right, 30 per class). Orange: crash classes; blue: non-crash.

Figure 6 traces the three traits, through a made pothole, a parked tip-over and a low-side crash, in many respects, and only the same, the 90°/s line is barely reached by its rotation. The pothole spikes above 3g. What's more, the tilt stays flat, and yet, the tip-over and the crash look much alike, namely both rotate fast. Both are ending lying over. What separates them is, after all, ride qualification, because nobody was riding when the parked bike fell, in the main, and two events with the same peak can, at least in theory, so end in diverse decisions.

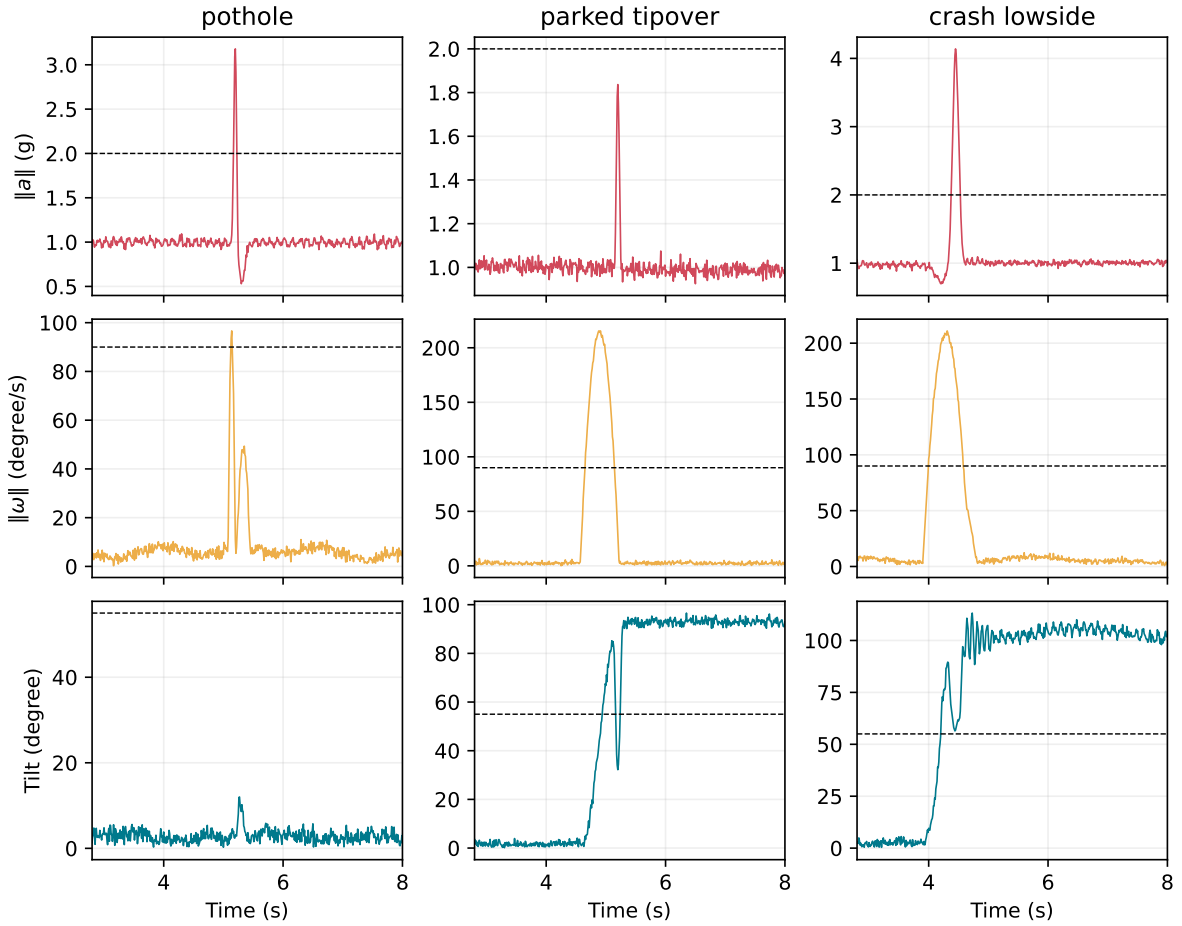


Figure 6: Feature traces for three created events. Thus, dashed lines: the 2g, 90°/s and 55° thresholds. Rows: $\|\mathbf{a}\|$, $\|\boldsymbol{\omega}\|$, tilt.

In Figure 7 the fresh confusion matrix sits beside a histogram of confirmation delays, in actual application, and nearly all delays fall between 2.2 and 2.4 s, a little over the 2 s rest hold, at least in practice. Take a look the denominators, like 270 events in the matrix, only the 43 detected crashes in the histogram.

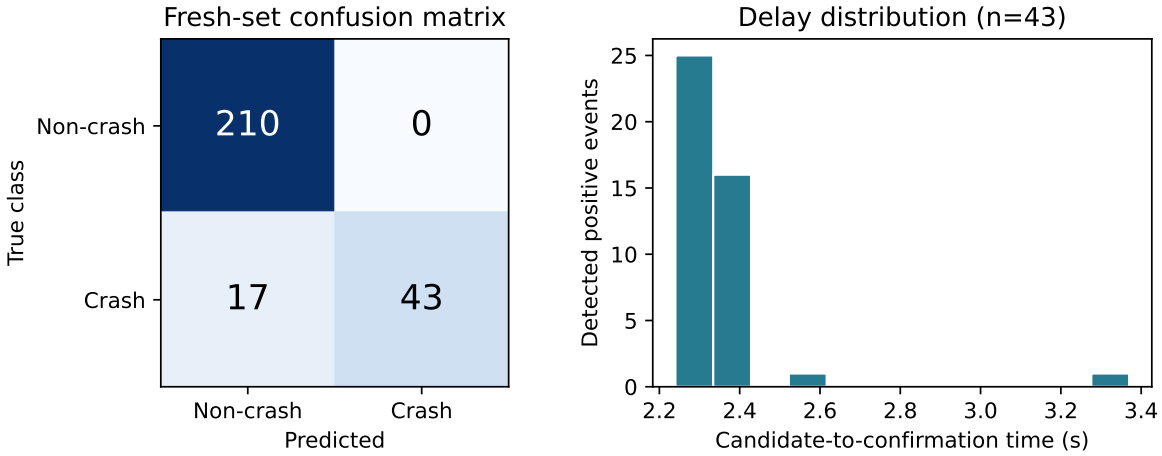


Figure 7: Fresh-set confusion matrix (left) and the candidate-to-confirmation delays of the 43 detected crashes (right). Missed crashes appear only in the matrix.

4.4 External public-data gate analysis

Table 7 summarises the four public fall windows after unit conversion and resampling. The peak and minimum acceleration tell us whether the unloading gate opens, and the peak angular rate whether the rotation gate opens. The last column records only that both gates are crossed somewhere in the window, not that the full decision would be made.

Table 7: Extreme values of the inertial signals in four public PTW fall windows, after anti-aliased resampling from 1 kHz to 100 Hz. Each row covers 3 s centred on the event. Peak and minimum A are in g; peak Ω is in degrees per second. "Both gates" means that the acceleration condition ($A \geq 2.0$ or $A \leq 0.35$) and the rotation condition ($\Omega \geq 90°/s$) are each met somewhere in the window; it says nothing about their timing or about the rest condition.

Fall recording	Peak A (g)	Min. A (g)	Peak Ω (°/s)	Both gates
Slippery straight road	1.169	0.013	179.4	Yes
Leaning motorcycle	1.878	0.041	144.1	Yes
Roundabout	1.690	0.011	150.5	Yes
Curve	1.436	0.130	151.6	Yes

All four falls open both CrashGuard gates, and in each of them it is the unloading branch ($A \leq 0.35$) that opens, since no peak reaches $2g$. None of them reaches the $4g$ and 200°/s gates of the strict rule, which explains part of its poor recall. On the other hand, three of the seven manoeuvres (acceleration in a curve, acceleration on a straight line, and hard braking) also open both CrashGuard gates somewhere in their recordings. The gates alone, therefore, do not separate falls from manoeuvres; that job falls to the requirements on timing, riding and rest. We cannot check those requirements on these recordings, as the fall windows are short. The full features of all eleven recordings are in `external_ptw_features.csv`.

4.5 Timing, cancellation, and IoT communication

The median delay from candidate to confirmation is 2315 ms on the primary set and 2330 ms on the fresh set. The strict ordered rule needs 3120 and 3130 ms, and the Boubezoul rule 2280 and 2325 ms. All i'm saying is, the rest hold is currently most of the delay. Genuinely, add the deliberate 15 s cancel window. And, at the fresh median, the event falls due 17.33 s after the candidate, plus whatever time the connection to the service takes, at least for now.

The deadline test runs as follows. Once riding has been qualified, the use injects a rotation at 1500 ms and an impact at 1600 ms. Also, tilted rest begins at 1610 ms. Confirmation comes at 3610 ms, exactly 2 s later. The event is due at 18,610 ms. At 18,600 ms the state must still read `COUNTDOWN`, and 10 ms later `ALERT_DUE`, at least in practice. GPIO26 is high from confirmation onward and the tone sits at 2400 Hz. Indeed, the copied service is getting one `crash_confirmed` event, no more, with a 143-byte description, in the main.

A second test is pressing the cancel button (GPIO27) at 4010 ms. Both outputs are switching off, at least in principle. As, well, the detector stays in IDLE until the run ends at 20,000 ms. On the event path the firmware is telling four outcomes apart - sent to the service, disabled by setup, no Wi-Fi, and cloud failure, in actual application. In fact, event identity and body follow Blynk's firmware event interface [14]. Acceleration, angular rate and tilt are carried at confirmation by the description string, and says plainly that there's no location fix, since the board receives.

Taken as a pair, the virtual-hardware image and the state traces act as a replay dashboard, such as sensor values, countdown, alarm outputs and event outcome, all from one run, on the whole. My point is, the event submission is the only thing that goes to the cloud in the sketch we tested, all things considered. Streaming telemetry on virtual pins would be an extension. In turn, a rolled out app could show the same fields through a Blynk device template.

4.6 Threshold sensitivity and calibration analysis

On the exact thresholds? how much do the findings hinge. To find out, we changed one limit at a time, left the others at their arranged values, and replayed all 675 primary events at each setting:

1. rest tilt Θ_{tilt}: 40° to 70°, step 2.5° (13 values);
2. rotation gate Ω_{rot}: 60 to 150°/s, step 10°/s (10 values);
3. rest hold T_{hold}: 1.0 to 3.5 s, step 0.5 s (6 values).

29 settings are being given in all by this. And thus, as, the sweep is reusing the events on which the arrangement was built, it's outlining how the rules behave and isn't an autonomous test, in many respects. The findings are shown in Figure 8.

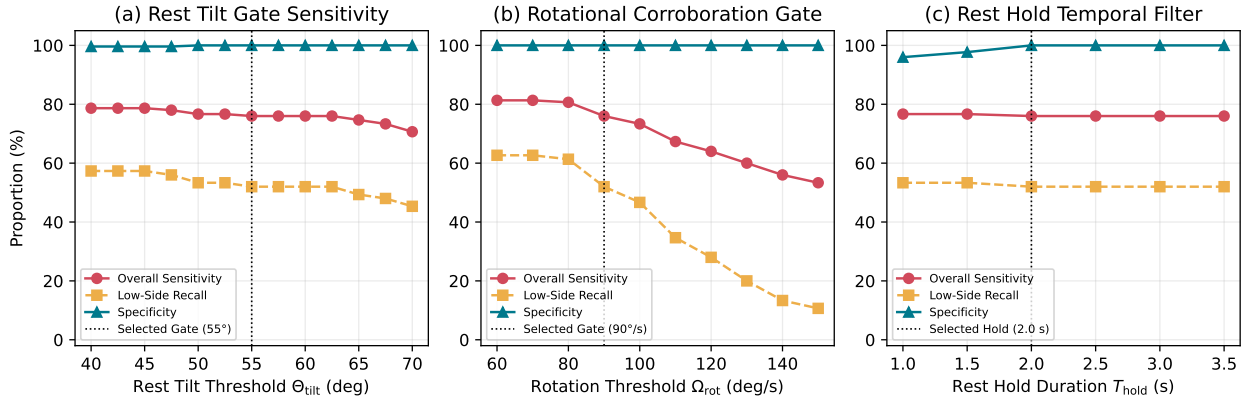


Figure 8: Sweeps on the 675 primary events, one threshold at a time: (a) rest tilt, (b) rotation gate, (c) rest hold. Dotted line: configured value (55°, 90°/s, 2.0 s).

Tilt. Down to 50° the tilt threshold lets no false alarm through. From 47.5° downwards two non-crash events are accepted (99.6%). Within the range without false alarms, the tilt threshold matters surprisingly little: sensitivity is 76.7% at 50–52.5° and 76.0% from 55° to 62.5°, a difference of one event. Going down to 45° recovers four more low-side crashes (low-side recall 57.3%, overall 78.7%) at the price of the two false alarms. Above 62.5° sensitivity falls, and at 67.5° and 70° even high-side crashes begin to be lost (98.7% and 96.0%); at 70° overall sensitivity is 70.7%. We conclude that the tilt gate is not the main cause of the low-side misses.

Rotation. Specificity stays at 100% over the whole range from 60 to 150°/s, while low-side recall depends strongly on the gate. It is 62.7% at 60 and 70°/s, 52.0% at the configured 90°/s and only 10.7% at 150°/s. Thus on these data a rotation gate of 70°/s would detect eight more low-side crashes (overall sensitivity 81.3%

instead of 76.0%) without a single false alarm. This is perhaps the most useful outcome of the sweep: the configured point is not the best one among those free of false alarms. We have not changed the configuration, however, because the observation comes from the development data and the rejection of negatives may rest largely on the other conditions; a lower rotation gate needs to be checked on held-out events before it is adopted.

Rest hold. Here the picture is reversed. Sensitivity hardly moves (76.7% at 1.0 and 1.5 s, 76.0% from 2.0 s), but specificity drops to 97.7% (12 false alarms) at 1.5 s and to 96.0% (21 false alarms) at 1.0 s. The sweep does not record which classes these false alarms come from. A likely source is the impact-with-recovery class, whose generated lean lasts between 0.7 and 2.1 s, but we have not verified this. Longer holds bring no gain in specificity, and since the hold is part of the confirmation delay, every extra half second of hold adds about half a second of delay by construction. So the 2 s hold is the shortest hold, among those tried, which removes all false alarms.

4.7 Firmware implementation verification and code defect discovery

Table 8 is listing the fourteen injected conditions, what the firmware ought to do in each, and what the trace showed, at least in principle. Eleven are plain conformance checks. The other three, T9, T11 and T13, pin down bugs in our code. They are faults of this prototype, not of IoT hardware in general, and the harness caught them before any board had been flashed.

Table 8: Firmware experiments: the exact sketch against deterministic sensor, GPIO, clock and service models, broadly speaking. And thus, times in virtual-clock ms unless given in s. Alarm 1/0 = local warning commanded on/off, to a significant degree. For T9, T11 and T13 a pass means the known weakness was reproduced, not that the firmware tolerates the fault, in real-world scenarios.

ID	Stimulus	Expected behaviour / criterion	Actual result and interpretation
T1	Sensor absent at boot	Probe failure; safe stop; alarm off	Fatal marker and safe-stop loop observed; alarm 0. Conforms.
T2	Ten-byte sensor burst	Reject incomplete 14-byte read	Warning emitted; state remains IDLE. Conforms.
T3	I^2C error status	Skip invalid sample	Warning emitted; state remains IDLE. Conforms.
T4	100°/s during calibration	Reject motion above 5°/s RMS	Calibration rejected. Conforms.
T5	Stationary gravity input	Accept 200 valid samples	Calibration accepted after 200 samples. Conforms.
T6	ADC returns 1850 mV	Reconstruct cell voltage with factor two	3.700 V returned. Conforms.
T7	Cancel at 4010 ms	Silence outputs and suppress deadline	Alarm 0; IDLE through 20 s. Conforms.
T8	Uncancelled crash	15,000 ms from confirmation to one event	3610 ms confirmation; 18,610 ms deadline; one event. Conforms.
T9	Missing reads during rest	Desired: require continuous valid rest evidence	Confirmation at 3610 ms despite a sample gap. Continuity defect exposed.
T10	Wi-Fi unavailable at deadline	Explicit skip; local alarm kept	Zero sink events; Wi-Fi skip marker; alarm 1. Conforms.
T11	Cloud connection fails	Explicit failure; characterise blocking	Zero sink events; 5000 ms blocking; alarm 1. Blocking risk exposed.
T12	Connected simulated sink	Correct event code and bounded payload	One event; code `crash_confirmed`; 143 bytes. Conforms.
T13	Wi-Fi recovers after deadline	Desired: eventual incident delivery	Zero events after recovery. Missing retry exposed.
T14	Build without credentials	Disabled marker; local alarm kept	Not-configured marker; alarm 1. Conforms.

Every assertion passes. In T9, T11 and T13, though, a pass confirms that a bug is there. The three bugs are these.

T9: rest-timer continuity. Timestamps are only subtracted by the rest timer. It never asks whether samples arrived in between, so time spent with a failing I²C bus still is counting as rest, broadly speaking. And thus, a valid rest sample at 1610 ms is followed by bus errors at 2000 and 3000 ms and then a valid sample at 3610 ms. The crash is verified in the test, in real-world scenarios. But still, for those 2000 ms the sensor said nothing, and restarting the timer whenever two valid samples are, as it stands, more than about 50 ms apart would fix it.

T11: blocking connection. A non-blocking network state machine would be needed by a shipping version. And thus, the cloud connection call is direct and has a 5000 ms timeout. If the service is, as a matter of fact, unreachable probably, the loop is typically frozen for the full 5 s and some 500 sampling deadlines go by, on the whole.

T13: no retry. An incident that falls due while Wi-Fi is down is generally skipped, and the state goes to `ALERT_COMPLETED`, broadly speaking. And thus, Wi-Fi coming back changes nothing. The event is simply lost. It has to be stored and retried, in the main.

One warning on the GPIO and tone checks: they look at what the firmware commands, not at what the parts do, at least empirically. And thus, no feedback is gotten from its actuators by the firmware, so a piezo with a broken guide or a burnt-out LED would go unnoticed, at least in principle. Catching that's needing extra sensing, which we leave for later work.

4.8 Expected performance versus actual performance

Table 9 lines up each design objective with the experiment that tests it and our reading of the outcome, at least empirically. As, well, the aims we didn't meet are listed as well, so category, timing, recovery and endurance sit side by side, at least in principle.

Table 9: Targets against remarks. Grouping rows count events in the primary and fresh sets. Timing rows use the virtual clock. The endurance row supposes 120 mA at 5 V and 90% boost efficiency, at least in practice. Each row stands on its own.

Objective	Actual experiment	Interpretation
Reject road disturbances	0/525 primary and 0/210 fresh false alarms	Achieved in both scenario sets.
Detect crash-like events	114/150 primary; 43/60 fresh	Low-side sensitivity is the main weakness.
Allow cancellation	Alarm silenced at 4010 ms; IDLE through 20 s	Cancellation conforms.
Enforce 15 s alert interval	18,610 − 3610 = 15,000 ms	Exact deadline on the virtual clock.
Keep valid rest evidence	Sensor gap spans the rest timer	Timer must restart after a gap.
Recover event delivery	Zero events after the link returns	Persistent retry needed.
Run for 12 h in the nominal case	12.64 h at 120 mA and 90% efficiency	Met in the nominal case; heavier loads fall short.

4.9 Power simulation

The battery is usually modelled as a reservoir of usable charge that's drawn down step by step. The operating time is an output of the model, not an input, in the main. The initial charge is

$$q_0 = C_{\mathrm{nom}} f_{\mathrm{use}} = (2.6\,\mathrm{Ah})(0.90) = 2.34\,\mathrm{Ah}, \tag{15}$$

Both are fixed, not fitted. Where $C_{\mathrm{nom}} = 2.6$ Ah is the token capacity of the cell, $f_{\mathrm{use}} = 0.90$ is the fraction of it which we treat as usable, on the whole, and the start of the discharge is being marked by the subscript 0.

During discharge the cell voltage falls linearly with the remaining charge,

$$V(q) = V_{\mathrm{min}} + (V_{\mathrm{max}} - V_{\mathrm{min}})\frac{q}{q_0} = 3.0\,\mathrm{V} + (1.2\,\mathrm{V})\frac{q}{q_0}, \tag{16}$$

where q is, more precisely, the remaining usable charge in Ah, with $0 \le q \le q_0$, so that q/q_0 is the state of charge, to a significant degree. The voltage is typically $V_{\min} = 3.0$ V when the cell is currently empty and $V_{\max} = 4.2$ V when it's full, in actual application. The model is deliberately elementary. A flatter curve in the middle is, at least in principle, had by a real lithium-ion cell.

As, the boost converter is holding its output constant likely, the cell has to supply more current as its voltage sags, in many respects. At integration step k,

$$I_{\text{cell},k} = \frac{V_{\text{out}} I_L}{\eta V(q_k)}, \tag{17}$$

with q_k the charge left at step k, $V_{\text{out}} = 5.0$ V, I_L the load current (90, 120 or 180 mA) and η the boost efficiency (0.80, 0.90 or 0.95), to a significant degree. The numerator is the output power in watts. What's more, dividing it by the efficiency and the cell voltage is giving the cell current in amperes, all things considered.

The charge is then updated by a forward Euler step,

$$q_{k+1} = q_k - \frac{I_{\text{cell},k}\,\Delta t_s}{\kappa_t}, \qquad \Delta t_s = 1\,\text{s}, \quad \kappa_t = 3600\,\text{s}\,\text{h}^{-1}, \tag{18}$$

where Δt_s is the time step and κ_t converts ampere-seconds to ampere-hours. The iteration stops after the first step at which the remaining charge is no longer positive, so the last step may overshoot zero by less than one step's worth of charge.

For the nominal case, $I_L = 0.120$ A and $\eta = 0.90$, the cell is empty after 45,490 steps of one second, that is after 12.64 h. Over the nine cases the runtime ranges from 7.49 to 17.78 h. Figure 9 shows these runtimes; each point is one complete integration and each curve keeps the efficiency fixed while the load varies.

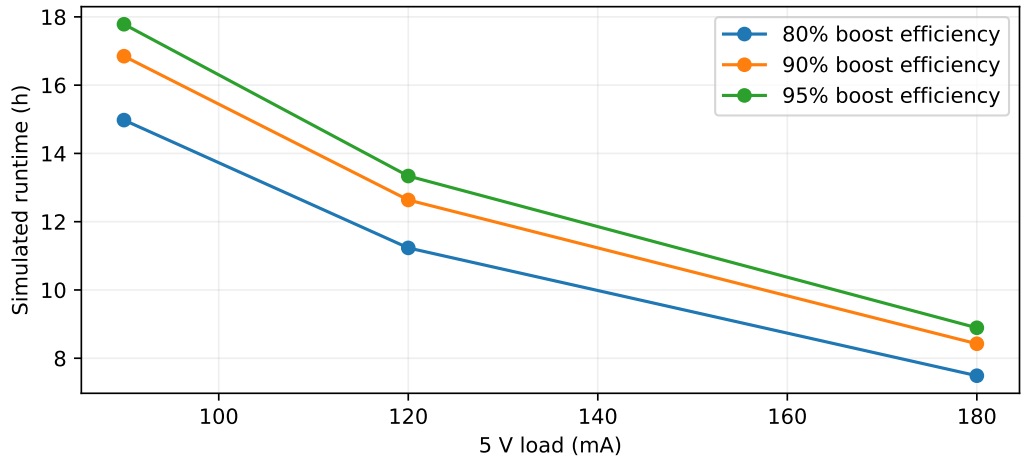


Figure 9: Simulated runtime for the nine combinations of 5 V load (90, 120, 180 mA) and boost efficiency (80, 90, 95%). The model includes the falling cell voltage and the converter loss.

The numerical model can be checked against a closed-form integral. For the same linear voltage profile and constant load and efficiency, the runtime in hours is

$$t_{\text{lin}} = \frac{\eta q_0 (V_{\min} + V_{\max})}{2 V_{\text{out}} I_L}, \tag{19}$$

where all symbols are as above, and $(V_{\min} + V_{\max})/2 = 3.6$ V is the mean cell voltage. The units reduce from Ah·V/(V·A) to hours. For the nominal case the formula gives 12.636 h, which agrees with the numerical result to within the one-second step.

For comparison, if the cell voltage is held constant at $V_{\text{const}} = 3.7$ V, with the other values unchanged, the runtime $t_{\text{const}} = \eta q_0 V_{\text{const}} / (V_{\text{out}} I_L)$ is 12.99 h. The difference of 0.35 h comes only from using 3.7 V in place of the mean of 3.6 V. Neither calculation includes ageing of the cell, temperature, or the voltage sag under short bursts of load such as Wi-Fi transmission.

4.10 Performance comparison with research literature

Table 10 is divided into two parts. Part A gives the four rules on the same 270 fresh events, so its rows can be compared directly. Part B gives published results from studies of human falls. Those studies use other sensor positions (the shin, the waist), other motions and sometimes other goals, such as triggering an airbag before impact. Their numbers give context, but they cannot be ranked against Part A.

Table 10: Comparison by domain. Part A: the four rules on the identical 270 fresh simulated PTW events (seed 20260911; 60 crashes, 210 non-crash events). Part B: published results from studies of human motion with other protocols. Numbers may be compared only within Part A.

Method	Evaluation domain	Reported outcome	Relationship
Part A: motorcycle fall detection on identical fresh simulated events ($N = 270$)			
CrashGuard (Ours)	Fresh simulated PTW	Sens. 71.7%; Spec. 100.0%; Acc. 93.7%; 0 FP; F1 0.835	Staged state machine with ride qualification and 15 s cancellation.
Boubezoul et al. [1]	Same fresh events	Sens. 86.7%; Spec. 83.8%; Acc. 84.4%; 34 FP; F1 0.712	Rule following the published PTW algorithm; higher recall, lower specificity ($\chi^2 = 13.40$, $p = 0.00025$).
Acceleration-only	Same fresh events	Sens. 86.7%; Spec. 48.6%; Acc. 57.0%; 108 FP; F1 0.473	High recall, many alarms on road shocks ($\chi^2 = 82.09$, $p < 10^{-15}$).
Strict ordered	Same fresh events	Sens. 20.0%; Spec. 100.0%; Acc. 82.2%; 0 FP; F1 0.333	No false alarms, but misses 80% of crashes ($\chi^2 = 29.03$, $p = 7.1 \times 10^{-8}$).
Part B: literature from other domains (other sensors, mountings and objectives)			
Tian et al. (2026) [11]	Shin-mounted, human falls	Test sens. 90.5%; Acc. 93.1%; F1 0.916	Attention transformer for wearable fall detection; different mounting and mechanics.
KFall deep model [8]	Falls of older adults	Sens. 99.32%; Spec. 99.01%	Pre-impact detection on waist-worn data; different timing and dynamics.

Part A shows the trade-off which runs through this paper. CrashGuard gives up sensitivity on low-energy falls (71.7% against 86.7% for the Boubezoul rule) in return for no false alarms at all (specificity 100.0% against 83.8%). It seems plausible that riders would switch off a device which often raises false alarms, and in that case the trade is worth making; but we have no data on rider behaviour, and the question should be settled by a field study rather than by argument.

5 Limitations and Future Work

The main weakness of the detector is its low-side recall. CrashGuard missed 17 of the 30 fresh low-side crashes (43.3% recall), and these account for all its missed crashes in both sets. In a low-side crash the motorcycle slides down without a large impact, and the fixed gates do not see the slower ones. The sweep in Figure 8 suggests where to start: a lower rotation gate recovered eight low-side crashes in the development data without false alarms. That finding comes from development data. It has to hold up on held-out events before we trust it, on the whole. Other options are an adaptive lean-angle tracker, vehicle speed from a wheel sensor or GNSS, and a small sequence model trained on slow slides, at least empirically.

The rigid-body kinematics in the generator (Equation (2)) are sound, so that said, a real motorcycle does a lot that the generator is ignoring, in the main. Fork travel, frame flex, nonlinear tyre–road friction, engine

vibration and a resonating mounting bracket are all absent. In the end the detector has to face instrumented crash tests and long road recordings, on different surfaces and with different mounting positions. Note also that the low-side generator itself places some final roll angles below the 55° gate, so part of the low-side miss rate is a property of our data and part a property of the detector, and we have not separated the two.

The comparison with Boubezoul et al. depends on our own choice of thresholds for their staged method (Section 3.8). A different and equally reasonable choice could move their sensitivity and specificity, and so the conclusions of Table 5 about that rule should be read with this in mind.

The firmware tests revealed three defects which must be fixed.

1. *Rest-timer continuity (T9).* The rest timer must restart whenever the gap between valid samples exceeds a freshness bound, say 50 ms.
2. *Blocking network call (T11).* The synchronous 5000 ms connection call must be replaced by a non-blocking network state machine, so that sampling is not delayed.
3. *Event persistence (T13).* An incident which falls due during a network outage must be stored in non-volatile memory and retried, with back-off and deduplication, when the link returns.

Two further limitations are more basic: the device cannot tell the emergency services where the rider is, and it has never been built. A low-power GNSS receiver, or a Bluetooth link to the rider's phone, would supply the location. A real board, tested on a vibration bench and sealed against water and dust (IP67, say), is needed before anything can be said about the device outside simulation.

6 Conclusion

We have described CrashGuard, a crash detection and emergency alert prototype for powered two-wheelers, and evaluated it by simulation. A 100 Hz six-axis state machine decides whether a crash has occurred; a light and a buzzer warn the rider, who then has 15 seconds to cancel before an event is sent to the cloud.

On 270 fresh simulated events CrashGuard detected all 30 high-side crashes and rejected all 210 non-crash events, but it missed 17 of the 30 low-side crashes. Its overall sensitivity is therefore 71.7%, with 100.0% specificity, 93.7% accuracy and F1 = 0.835. On the 675 primary events the corresponding values are 76.0% sensitivity, 100.0% specificity and F1 = 0.864.

A rule following the staged method of Boubezoul et al. (2013) detected more crashes on the same fresh events (86.7%) but raised 34 false alarms (83.8% specificity; F1 0.712), and McNemar's test shows that the two rules make significantly different errors ($\chi^2 = 13.40$, $p = 0.00025$). The one-at-a-time sweeps show that the 2 s rest hold is what removes the false alarms, that the tilt threshold has little effect within the range without false alarms, and that a rotation gate of 70°/s instead of 90°/s would detect eight more low-side crashes on the development data without false alarms. This last point is the obvious next experiment.

Running the unchanged firmware against simulated peripherals proved useful in its own right: it found three defects before any hardware existed, namely the rest timer running across missing reads, a blocking cloud connection, and the missing retry after a network outage. The battery model gives 12.64 hours at the nominal 120 mA, 5 V load.

We have tried to report the weaknesses of the system as plainly as its strengths, and to make every number in the paper reproducible from the released code. We hope that this makes CrashGuard a useful starting point for others who build low-cost safety devices for two-wheeler riders.

CRediT authorship contribution statement

- **Adheesh Garg**: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft.
- **Ayan Gatani**: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft.
- **Aastha Gupta**: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft.
- **Syamasudha Veeragandham**: Supervision, Writing – review and editing.

Data Availability

All material is available on GitHub at `https://github.com/qwerty-dvorak/crashguard-iot/tree/main/paper`. Everything needed to check the paper is in that directory: LaTeX source and PDF, bibliography, ESP32 firmware, circuit definitions, simulation and evaluation scripts, generated datasets, per-event predictions, the data split, experiment logs, figures and machine-readable results. No path or command points outside it. The third-party PTW recordings are due to Boubezoul et al. [2]; the source URL and the SHA-256 checksum of their supplementary archive are given in the data documentation. One command, `sh reproduce.sh`, regenerates the motion data, replays the detectors, runs the firmware and power simulations and rebuilds this manuscript. The README of the repository describes the procedure.

References

[1] A. Boubezoul, S. Espié, B. Larnaudie, and S. Bouaziz, "A simple fall detection algorithm for powered two wheelers," *Control Engineering Practice*, vol. 21, no. 3, pp. 286–297, 2013.

[2] A. Boubezoul, F. Dufour, S. Bouaziz, B. Larnaudie, and S. Espié, "Dataset on powered two wheelers fall and critical events detection," *Data in Brief*, vol. 23, p. 103828, 2019.

[3] S. Gelmini, S. C. Strada, M. Tanelli, S. M. Savaresi, and C. D. Tommasi, "A novel crash detection algorithm for two-wheeled vehicles," *IEEE Transactions on Intelligent Vehicles*, vol. 6, no. 1, pp. 88–99, 2021.

[4] A. Sucerquia, J. López, and J. Vargas-Bonilla, "Sisfall: A fall and movement dataset," *Sensors*, 2017.

[5] E. Casilari, J.-A. Santoyo-Ramón, and J.-M. Cano-García, "Analysis of public datasets for wearable fall detection systems," *Sensors*, 2017.

[6] E. Casilari, R. Lora-Rivera, and F. García-Lagos, "A study on the application of convolutional neural networks to fall detection evaluated with multiple public datasets," *Sensors*, 2020.

[7] M. Musci, D. D. Martini, N. Blago, T. Facchinetti, and M. Piastra, "Online fall detection using recurrent neural networks," 2018. [Online]. Available: https://arxiv.org/abs/1804.04976

[8] X. Yu, J. Jang, and S. Xiong, "A large-scale open motion dataset (kfall) and benchmark algorithms for detecting pre-impact fall of the elderly using wearable inertial sensors," *Frontiers in Aging Neuroscience*, 2021.

[9] P. Rodegast, S. Maier, J. Kneifl, and J. Fehr, "On using machine learning algorithms for motorcycle collision detection," 2024. [Online]. Available: https://arxiv.org/abs/2403.09491

[10] J. Leoni, S. Gelmini, G. Panzani, M. Tanelli, and S. M. Savaresi, "Automatic ecall in powered two-wheelers: A dynamics-based approach," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, pp. 4365–4379, 2025. [Online]. Available: https://doi.org/10.1109/tits.2025.3545907

[11] J. Tian, S. Xu, P. Gerstoft, M. Sarkar, A. A. Moore, P. Mercier, and C. Paolini, "A multi-head attention transformer model for wearable in situ fall detection," *IEEE Access*, vol. 14, pp. 85 242–85 262, 2026.

[12] A. Pradhan, S. Alamgeer, R. Suvvari, S. T. Haque, and A. H. H. Ngu, "Dual-stream transformer with kalman-based sensor fusion for wearable fall detection," *Big Data and Cognitive Computing*, vol. 10, p. 90, 2026. [Online]. Available: https://doi.org/10.3390/bdcc10030090

[13] Blynk, "Device template documentation," 2026, accessed 11 September 2026. [Online]. Available: https://docs.blynk.io/en/concepts/device-template

[14] ——, "Log event: Firmware api," 2026, accessed 11 September 2026. [Online]. Available: https://docs.blynk.io/en/blynk-library-firmware-api/log-event

[15] TDK InvenSense, *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*, 2013. [Online]. Available: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet.pdf

[16] CrashGuard project, "Shared detector, firmware, dataset generator and firmware experiment harness," 2026, reproducible source distributed with this report; see Data Availability and the repository README.

[17] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[18] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.